

Web Application Penetration Testing

Proof-of-Concept (PoC) Report

Project Title: Web Application Penetration Testing (OWASP Top 10)
Target Application: DVWA (Damn Vulnerable Web Application)
Testing Environment: Kali Linux

1. Executive Summary

This report presents the results of a security assessment conducted on the **Damn Vulnerable Web Application (DVWA)**. The objective of the assessment was to identify and demonstrate common web application vulnerabilities based on the **OWASP Top 10 security risks**.

During testing, several critical vulnerabilities were successfully identified and exploited in a controlled environment. These vulnerabilities include **SQL Injection, Brute Force Authentication, Cross-Site Scripting (XSS), Sensitive Data Exposure, Command Injection, and File Inclusion**.

The findings demonstrate how improper input validation, weak authentication mechanisms, and insecure configurations can allow attackers to gain unauthorized access, extract sensitive data, and potentially compromise the entire server. Organizations should implement secure coding practices, proper authentication mechanisms, and strong cryptographic protections to mitigate such vulnerabilities.

Source:

OWASP Top 10 Project <https://owasp.org/www-project-top-ten/>

2. Testing Methodology

The penetration testing process followed a structured methodology commonly used in security assessments.

1. Information Gathering

Initial reconnaissance was conducted to understand the structure and functionality of the application.

2. Vulnerability Identification

Various modules within DVWA were tested for common vulnerabilities aligned with OWASP Top 10.

3. Exploitation

Proof-of-Concept attacks were performed to demonstrate the impact of each vulnerability.

4. Impact Analysis

The potential risks associated with each vulnerability were analyzed.

5. Remediation Recommendations

Security best practices were suggested to mitigate the identified vulnerabilities.

Reference:

OWASP Testing Guide <https://owasp.org/www-project-web-security-testing-guide/>

3. Testing Environment

Component	Details
Operating System	Kali Linux

Target Application	DVWA
Web Server	Apache
Database	MySQL
Testing Tools	Burp Suite, Browser Developer Tools
Network Setup	Localhost environment

The testing was conducted in a **controlled lab environment** to ensure no real systems were affected.

4. Vulnerability Findings

4.1 SQL Injection

Vulnerability Details

Parameter	Value
OWASP Category	A05 – Injection
Severity	Critical
Affected Module	SQL Injection
Risk Level	High

SQL Injection occurs when an application fails to properly sanitize user input before executing database queries. Attackers can manipulate database queries to retrieve unauthorized data.

Source: https://owasp.org/www-community/attacks/SQL_Injection

Proof of Concept

Step 1: Confirm Vulnerability

Input:

'

Result:

A database error appeared, indicating unsanitized input.

Step 2: Identify Number of Columns

Payloads used:

1' ORDER BY 2#

1' ORDER BY 3#

Result:

Error appears at column 3 → query contains **2 columns**.

Step 3: Extract Database Information

1' UNION SELECT @@version, database()#

Result:

Database version and name were successfully retrieved.

Step 4: Dump Credentials

1' UNION SELECT user, password FROM users#

Result:

Username and hashed passwords were extracted from the database.

Impact

An attacker could:

- Access sensitive database information
- Extract credentials
- Modify or delete records
- Gain administrative access

Remediation

Use **Prepared Statements** or **Parameterized Queries**.

Example secure code:

```
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = :id');  
$stmt->execute(['id' => $id]);
```

4.2 Brute Force Authentication

Vulnerability Details

Parameter	Value
OWASP Category	A07 – Authentication Failures
Severity	High
Affected Module	Brute Force Login

The application does not implement account lockout or rate limiting, allowing automated password guessing attacks.

Source: https://owasp.org/www-community/attacks/Brute_force_attack

Proof of Concept

Tool Used

Burp Suite Intruder

Attack Steps

1. Intercept login request.
2. Send request to Intruder.
3. Set attack type to **Cluster Bomb**.
4. Use username and password wordlists.
5. Launch attack.

Example discovered credential:

admin : password

Impact

Attackers can gain unauthorized access to user accounts and potentially obtain administrative privileges.

Remediation

- Implement account lockout
- Enforce strong password policies
- Add CAPTCHA
- Enable Multi-Factor Authentication (MFA)

Reference:

OWASP Authentication Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

4.3 Cross-Site Scripting (XSS)

Vulnerability Details

Parameter	Value
OWASP Category	Injection
Severity	High

Cross-Site Scripting allows attackers to inject malicious JavaScript that executes in a victim's browser.

Source: <https://owasp.org/www-community/attacks/xss/>

Types Demonstrated

Reflected XSS

Payload:

```
<script>alert('Reflected XSS')</script>
```

Result:

Script executes immediately in the browser.

Stored XSS

Payload:

```
<script>alert('Stored XSS')</script>
```

Result:

Script stored in database and executed for all visitors.

DOM Based XSS

Payload:

```
?default=English<script>alert(document.cookie)</script>
```

Result:

JavaScript executed in client-side DOM.

Impact

Attackers can:

- Steal session cookies
- Hijack user sessions
- Redirect users to malicious sites
- Perform phishing attacks

Remediation

- Validate user input
- Encode output
- Implement Content Security Policy (CSP)

Reference:

Mozilla Web Security Guide <https://developer.mozilla.org/en-US/docs/Web/Security>

4.4 Sensitive Data Exposure

Vulnerability Details

Parameter	Value
OWASP Category	A04 – Cryptographic Failures
Severity	Medium

The application exposes sensitive system information and uses weak password hashing.

Proof of Concept

Accessed:

<http://localhost/phpinfo.php>

This revealed:

- PHP version
- Server configuration
- System information

Extracted password hash:

5f4dcc3b5aa765d61d8327deb882cf99

Using CrackStation, the hash revealed the password:

Password

Source: <https://crackstation.net/>

Impact

Attackers can perform **offline password cracking** and exploit system vulnerabilities.

Remediation

- Remove debug files
- Use strong hashing algorithms (bcrypt or Argon2)
- Enforce HTTPS encryption

Reference:

OWASP Password Storage Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

4.5 Command Injection

Vulnerability Details

Parameter	Value
OWASP Category	Security Misconfiguration
Severity	Critical

User input is directly executed as an OS command.

Source: https://owasp.org/www-community/attacks/Command_Injection

Proof of Concept

Payload:

127.0.0.1; cat /etc/passwd

Result:

The server executed the command and displayed system user information.

Impact

Attackers can execute arbitrary commands on the server and gain full system access.

Remediation

- Validate user input
- Avoid shell execution functions
- Use secure system APIs

4.6 File Inclusion (Local File Inclusion)

Vulnerability Details

Parameter	Value
OWASP Category	Broken Access Control
Severity	High

The application allows attackers to include unauthorized files.

Source: https://owasp.org/www-community/attacks/Path_Traversal

Proof of Concept

URL:

`http://localhost/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd`

Result:

The contents of the Linux `/etc/passwd` file were displayed.

Impact

Attackers can read sensitive system files and application source code.

Remediation

- Implement strict file whitelisting
- Disable URL inclusion
- Validate file paths

5. Conclusion

The penetration testing assessment successfully identified multiple vulnerabilities aligned with the **OWASP Top 10 security risks**. These vulnerabilities demonstrate how insecure coding practices and weak authentication mechanisms can expose web applications to serious attacks. Implementing proper input validation, strong authentication mechanisms, secure cryptographic practices, and server hardening can significantly reduce the risk of exploitation.

Regular security assessments and adherence to OWASP security guidelines are essential for maintaining a secure web application environment.

Source: OWASP Top 10 <https://owasp.org/www-project-top-ten/>